7.1 Стандартный пакет для логирования

Ключевые моменты

- Пакет log является частью стандартной библиотеки Go и не требует установки дополнительных зависимостей.
- Он поддерживает базовые функции логирования, такие как вывод сообщений с метками времени, но не имеет встроенных уровней логов (например, INFO, ERROR).
- Логи можно перенаправлять в файлы или другие потоки, а также настраивать с помощью флагов и префиксов.

Основы использования

Пакет log предоставляет функции, такие как Println, Printf, Fatal и Panic, которые используют предопределённый логгер для вывода сообщений в stderr с меткой времени.

```
package main
import "log" func main() {

   str := "Gophers"
   log.Printf("Hello %s\n", str)
   log.Println("Hello developers!")

   // Следующие строки завершат программу или вызовут панику
   // log.Fatal("Hello user, Testing Fatal error\n")
   // log.Panic("Hello user! This is how you log\n")
}
```

Логирование в файл

Вы можете перенаправить вывод логов в файл, используя log.SetOutput. Это полезно для сохранения логов для последующего анализа.

```
package main

import (
   "log"
   "os"
)

func main() {
   file, err := os.OpenFile("logs.txt", os.O_APPEND|os.O_CREATE|os.O_WRONLY, 0666)
   if err != nil {
      log.Fatal(err)
   }

   log.SetOutput(file) log.SetPrefix("INFO: ")
   log.SetFlags(log.Ldate | log.Ltime )

   log.Println("Hello world!")
}
```

Логирование в буфер

Логи можно записывать в буфер для временного хранения или обработки, например, для тестирования или передачи в другую систему.

```
package main

import (
   "bytes"
   "fmt"
   "log"
)

func main() {
   var buf bytes.Buffer
   logger := log.New(&buf, "logger: ",
   log.Lshortfile) logger.Print("Hello,
   log file!")
   fmt.Print(&buf)
}
```

Пользовательские логгеры

Создание нескольких логгеров с разными префиксами позволяет имитировать уровни логирования, такие как INFO, WARNING и ERROR.

```go
package main

 import (
     "log"
     "os"
)

var (
    InfoLogger    *log.Logger
    WarningLogger *log.Logger
    ErrorLogger   *log.Logger
)

func init() {
    file, err := os.OpenFile("logs.txt", os.O_APPEND|os.O_CREATE|os.O_WRONLY, 0666)
    if err != nil {
        log.Fatal(err)
    }

    InfoLogger = log.New(file, "INFO: ", log.Ldate|log.Ltime|log.Lshortfile)
    WarningLogger = log.New(file, "WARNING: ", log.Ldate|log.Ltime|log.Lshortfile)
    ErrorLogger = log.New(file, "ERROR: ", log.Ldate|log.Ltime|log.Lshortfile)
}

func main() {
    InfoLogger.Println("Starting the application...")
    WarningLogger.Println("There is something you should know about")
    ErrorLogger.Println("Something went wrong")
}
```

Рекомендации

- Используйте стандартный пакет log для простых приложений или прототипов, где не требуется сложная структура логов.
- Для имитации уровней логирования создавайте пользовательские логгеры с разными префиксами.

Обработка ошибок с логированием

Если вы решите обработать ошибку, записав ее в лог, по определению это больше не ошибка — вы ее обработали. Акт обработки ошибки обрабатывает ошибку, поэтому больше не следует обрабатывать ее как ошибку.

```
err := someFunc()
if err != nil {
   log.Error("Error",err)
   return err
}
```

```
if err := someFunc();err != nil {
   log.Infof("Info: %v", err)
   someOperation()
}
```